

Documentação C4 Model

Loja API - Cliente, Produto e Pedido

Arquitetura de uma API REST em Java Spring Boot, com persistência em banco H2 local, organizada no padrão MVC.

Níveis apresentados:

C1 Contexto do sistema

C2 Containers da solução

C3 Componentes internos da API

C4 Código / classes principais

1. Introdução

O C4 Model (Context, Container, Component, Code) descreve a arquitetura de software em quatro níveis, do mais amplo para o mais detalhado. Esta documentação registra a arquitetura da Loja API, um sistema simples de gestão de clientes, produtos e pedidos.

O objetivo do projeto é entregar uma API REST objetiva, sem microsserviços ou camadas extras. A aplicação segue o padrão MVC: os Controllers recebem as requisições HTTP, os Services aplicam regras básicas e os Repositories persistem os dados no H2.

Escopo do sistema

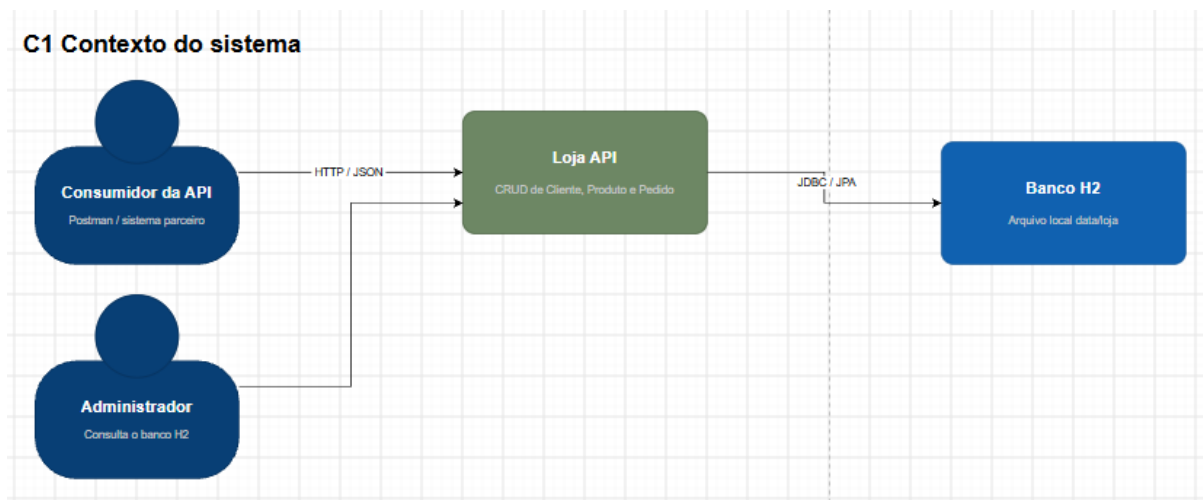
Três entidades com CRUD completo, busca por nome e contagem de registros: Cliente (id, nome, email), Produto (id, nome, preço) e Pedido (id, data, cliente, produto). Um pedido relaciona um cliente a um produto. Os dados ficam em um arquivo H2 local (data/loja).

Decisões de arquitetura

- Monolito simples: um único processo Spring Boot atende todas as rotas.
- API REST apenas (JSON). Sem autenticação, filas ou cache, para manter o foco no CRUD.
- H2 em arquivo local, suficiente para desenvolvimento e demonstração.
- JPA/Hibernate com ddl-auto=update, sem migrations complexas.
- Separacao MVC: Controller não acessa o banco; Repository não contém regra de negócio

2. C1 - Diagrama de Contexto

O nível 1 mostra o sistema como uma caixa única e quem interage com ele: o consumidor da API (Postman ou sistema parceiro), o administrador que consulta o console H2 e o banco de dados.

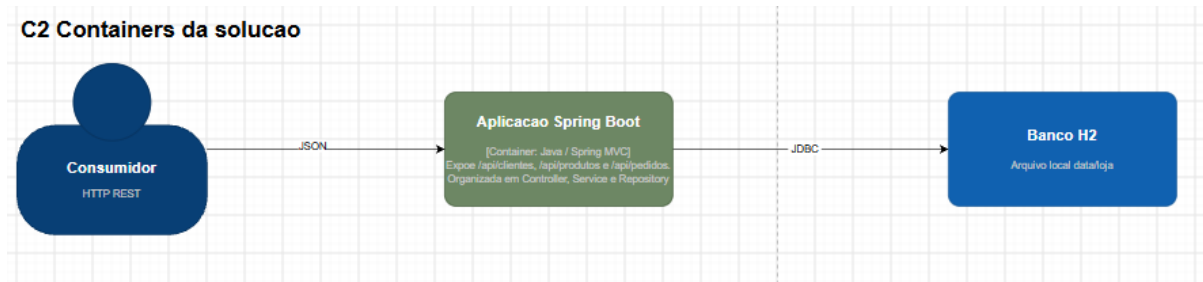


Relacionamentos:

- O consumidor envia requisições REST (GET, POST, PUT, DELETE) para /api/clientes, /api/produtos e /api/ pedidos.
- A Loja API lê e grava clientes, produtos e pedidos no H2.
- O administrador acessa o console H2 em /h2-console para inspecionar as tabelas

3. C2 - Diagrama de Containers

O nível 2 abre o sistema e mostra os containers de execução: a aplicação Spring Boot e o banco H2. Ambos rodam na máquina local.



Containers

Aplicação Spring Boot: único container de software da solução. Sobe na porta 8080, recebe as requisições, aplica as regras e persiste os dados. Contém os pacotes controller, service, repository e model.

Banco H2

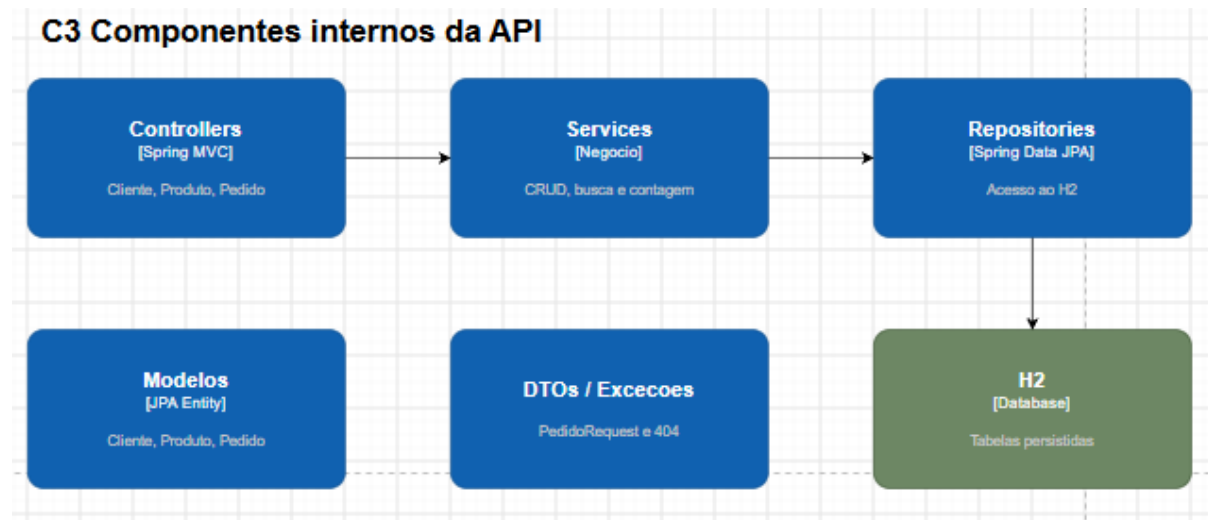
Container de dados. Usa a URL `jdbc:h2:file:./data/loja`. As tabelas **CLIENTE**, **PRODUTO** e **PEDIDO** são criadas automaticamente pelo JPA. O console web fica em `/h2-console`.

Comunicação

O consumidor fala **HTTP/JSON** com a aplicação. A aplicação fala **JDBC** com o **H2**, mediado pelo **Spring Data JPA / Hibernate**. Não há fila, gateway, autenticação nem cache.

4. C3 - Diagrama de Componentes

O nível 3 detalha os componentes dentro da aplicação Spring Boot, seguindo o MVC pedido no enunciado.



Controller

Ponto de entrada HTTP. Mapeia as rotas `/api/*`, valida o JSON e devolve status 200, 201, 204 ou 404. Não acessa o banco.

Service

Orquestra o CRUD, no pedido, busca Cliente e Produto pelos IDs informados e preenche a data automaticamente.

Repository

Interfaces Spring Data JPA. Além do CRUD padrão, oferecem busca por nome (`ContainingIgnoreCase`) e `count()`.

Model

Entidades JPA. Pedido possui `@ManyToOne` para Cliente e Produto (um pedido liga um cliente a um produto).

Tratamento de erro

`RecursoNaoEncontradoException` vira HTTP 404. Falhas de validação viram HTTP 400.

5. C4 - Codigo (classes principais)

O nível 4 mostra as classes concretas. A relação de domínio é: 1 Cliente tem N Pedidos; 1 Produto tem N Pedidos; cada Pedido aponta para 1 Cliente e 1 Produto.



Classes da aplicação

ClienteController: GET/POST/PUT/DELETE /api/clientes

ProdutoController: GET/POST/PUT/DELETE /api/produtos

PedidoController: GET/POST/PUT/DELETE /api/pedidos

ClienteService / ProdutoService / PedidoService: Regras de CRUD, busca e contagem

ClienteRepository / ProdutoRepository / PedidoRepository: JpaRepository + findByNome

PedidoRequest: DTO de entrada com clienteId e produtoId

TratamentoExcecao: Mapeia 404 e erros de validacao

6. Rotas e persistência

API REST

GET /api/{recurso} - lista

GET /api/{recurso}/{id} - busca por ID

GET /api/{recurso}/nome/{nome} - busca por nome (clientes e produtos)

GET /api/pedidos/cliente/{nome} - pedidos pelo nome do cliente

GET /api/pedidos/produto/{nome} - pedidos pelo nome do produto

GET /api/{recurso}/count - quantidade de registros

POST /api/{recurso} - cria (201)

PUT /api/{recurso}/{id} - atualiza

DELETE /api/{recurso}/{id} - remove (204)

Banco H2

URL: jdbc:h2:file:./data/loja

Usuario: sa

Senha: (vazia)

Console: <http://localhost:8080/h2-console>

Tabelas: CLIENTE, PRODUTO, PEDIDO